PWQ-VoxMercatus-AI

Project Card: PWQ-VM-AI V3S (Vox Mercatus AI)

Project Title: PWQ-VM-AI – Utilizing a Custom Target-Aware Transformer Neural Network to map emotional intensity as a leading indicator of a product's success.

I. **Project Overview.** PWQ-VM-AI is an advanced machine-learning project in the field of computational finance aimed at extracting and quantifying the "Voice of the Market" (Vox Mercatus). While traditional sentiment analysis systems often rely on simple word counting or polarity classification, this project employs a 4-Layer Target-Aware Transformer, with Neutralized NRoPE managing positional and token embedding, Custom FFN (Feed-Forward Network) Architecture which enriches processing data, and many more enhancements, to analyze the intensity of emotional expression in technological and product-related discussions on platforms such as Reddit, Twitter etc.

II. **Technical Architecture.** At the core of PWQ-VM-AI lies a 4-layer Transformer Network, built from scratch using PyTorch library and Python Programming Language. Unlike older AI models, this architecture is capable of understanding the sentiment in a sentence, and to whom it is being directed.

• Multi-Head NRoPE-Enhanced Self-Attention: Allows the model to understand relationships between words and phrases, whilst doing so, with ability of multiple perspective views simultaneously, which improves detecting advanced emotions such as sarcasm, irony, and technological nuance. ***Note, that in future versions, the main focus is to improve detection of these emotions specifically, as stated in point VIII.**

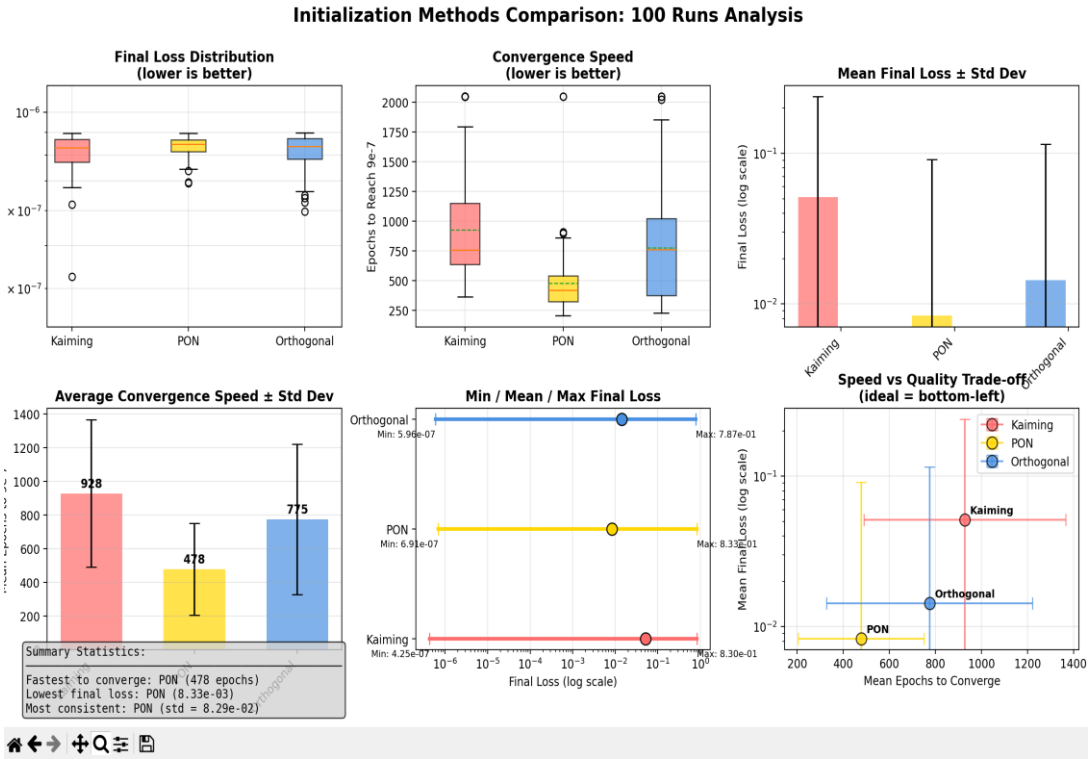
• Target-Specific Average Pooling: In standard transformers for sentiment analysis, the Global Average Pooling (GAP) would be applied over all tokens (the entire sentence), to extract the sentiment of the entire text. However PWQ-VM-AI, is using Target-Specific Average Pooling, which averages only the target tokens; thus making PWQ-VM-AI, fully Target-Aware.

```
# Calculate Average
target_token_counts = mask.sum(dim=1, keepdim=True).clamp(min=1)
globalAveragePooled = target_vectors.sum(dim=1) / target_token_counts
```

• Adaptive Dropout Layers: Dropout Layers, are typically used to teach neural network not to rely on any single neuron while inferencing, by purposefully "killing" (multiplying by zero) x% of its values, making the network having to optimize its weights in different combinations, which help in generalization (ability to function with dataset-exclusive data). However, during engineering of

PWQ-VM-AI, it was discovered that a much better way to use Dropout was to make that x% a learnable parameter (a value that optimizes during backpropagation), meaning each layer and each weight affected by Dropout, can have its own % of kill-rate, which improved PWQ-VM-AI final accuracy from 94.07% to 97.87%!

• Perturbated Ortho-Normalized Initialization: A very standard initialization technique for neural networks is Orthogonal, which combines Orthogonality and Normalization of vectors, which makes their distance between each other always 1 and the angle to 90deg. However, after studying: QR Decomposition, Householder Reflections, LDL/LU/PLU Decompositions, and more, a flaw was found. Sometimes that perfect symmetry, prohibits the network from being able to explore new options, and while that symmetry also causes no redundancy which means the network has maximum expressivity. However, after countless tests, and benchmarking, the result is that, if you add a very tiny "perturbator" matrix to the Q decomposed one (Q is the ortho-normalized matrix), after 100 runs on a 4-layer model, PWQ's Perturbated Ortho-Normalized initialization demonstrates superior convergence speed, exceptional stability, and near-optimal final loss – statistically outperforming both Kaiming and Orthogonal initializations. Which is exactly the technique behind Perturbated Ortho-Normalized Initialization, or PON.




```
@staticmethod
def pon(w: torch.Size, layers: int, mean:float=0.0, std:float=1.0, bias_return:bool=False,
perturb_bias:bool=True, debug_prints:bool=False) -> Union[torch.nn.Parameter,
Tuple[torch.nn.Parameter, torch.nn.Parameter]]:
```

```
    """
    Perturbated Ortho-Normalized method which returns torch.nn.Parameter object which is
the initialized weight matrix,
    and the initialized bias matrix as the perturbator matrix.
    """
    debug = DebugPrint(debug_prints)
    debug._log(f"Current Task: size-{w}, layers-{layers}")
    _in, _out = w
    in_bigger_out = True if _in > _out else False
    debug._log(f"Initializing PON V3")
    A = torch.normal(mean, std, size=(_in, _out)) if in_bigger_out else torch.normal(mean,
std, size=(_out, _in))
    pwq_poni = PWQ_PON_V_3_0(matrix=A)
    debug._log(f"Constructed Initial Matrix")
    q, r = pwq_poni.hhrl_(A)
    debug._log(f"QR Decomposition Complete")
    q_c, r_c = pwq_poni.qrec_(q, r, conv_its=6)
    q_c, _ = pwq_poni.qrec_(q_c, r_c, conv_its=1)
    q_c_slcd = q_c[:_out, :].T if in_bigger_out else q_c[:_in, :]
    debug._log(f"QR Error Correction Complete")
    q_ci, pbm = pwq_poni.init(q_c_slcd, _out, layers, perturb_bias=perturb_bias)
    debug._log(f"Perturbating Q Complete")
    if bias_return:
        debug._log(f"Returning weight of shape {q_ci.shape}, and bias of shape
{pbm.shape}")
        return torch.nn.Parameter(q_ci).to("cuda"), torch.nn.Parameter(pbm).to("cuda")
    return torch.nn.Parameter(q_ci).to("cuda")
```

III. Scientific Objective and Hypothesis.

•PWQ-VM-AI's scientific objective: to investigate the correlation between emotional intensity in social media discourse and real-world product success. The project seeks to determine whether a Target-Aware Transformer AI model can identify extreme enthusiasm or rejection before these emotions are reflected in product sales and profit.

•PWQ-VM-AI's hypothesis: Fluctuations in the PWQ-VM-AI Barometer observed on social platforms suggest that collective social emotions precede product outcomes. Specifically, a significant lean towards positive or negative emotional sentiment serve as a precursor to increased product sales / popularity.

• Hypothesis Results: After evaluating 3 products - iPhone 17 Pro Max, NVIDIA RTX 5090, and Avatar 3 – Fire and Ash, from 2 different aspects: 1. Based off of a non-biased / objective method of evaluation of product's success using three distinct areas: predecessor comparison (units sold), spendings recovery (>85% in 3 month period from release date), and a highly-reliable multi-source average top leaderboard standings (>top 5).

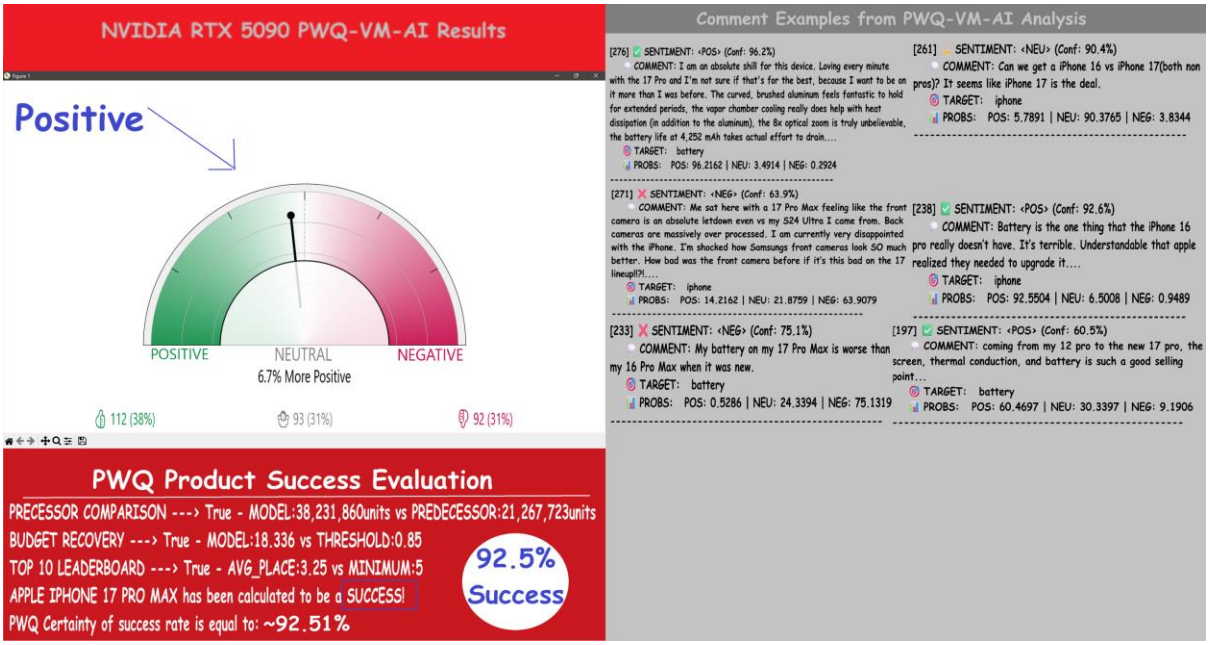
```
def PredecessorComparison(self, model_sales: Union[int, float], model_price: Union[int,
float], predecessor_sales: Union[int, float], predecessor_price: Union[int, float]) ->
```

```
Tuple[bool, float]:
    bool = model_sales/model_price > predecessor_sales/predecessor_price
    ratio = 100*((model_sales/model_price) / (predecessor_sales/predecessor_price) -1)
    final_ratio_prctng = ratio/3 if ratio < 100 else 33.3334
    return bool, final_ratio_prctng

def BudgetRecoveryFirstQuarter(self, total_budget_used: Union[int, float],
total_sales_first_quarter: Union[int, float], superiority_threshold:float=0.85) ->
Tuple[bool, float]:
    bool = total_sales_first_quarter/total_budget_used > superiority_threshold
    ratio = (total_sales_first_quarter/total_budget_used/2*superiority_threshold)**2
    final_ratio_prctng = ratio if ratio < 33.334 else 33.334
    return bool, final_ratio_prctng

def Top10LeaderboardAppearence(self, top10avg_place:Union[int,float], leaderboard_min: int
= 5) -> Tuple[bool, float]:
    if top10avg_place > 50:
        print("PRODUCT NOT IN TOP 50")
        return False, 0
    leaderboard_bool = top10avg_place <= leaderboard_min
    base_prctng = 3.334
    multiplier = 10*base_prctng - (top10avg_place-1)*base_prctng
    final_ratio_prctng = multiplier if 33.34 >= multiplier > 0 else 0
    return leaderboard_bool, final_ratio_prctng
```

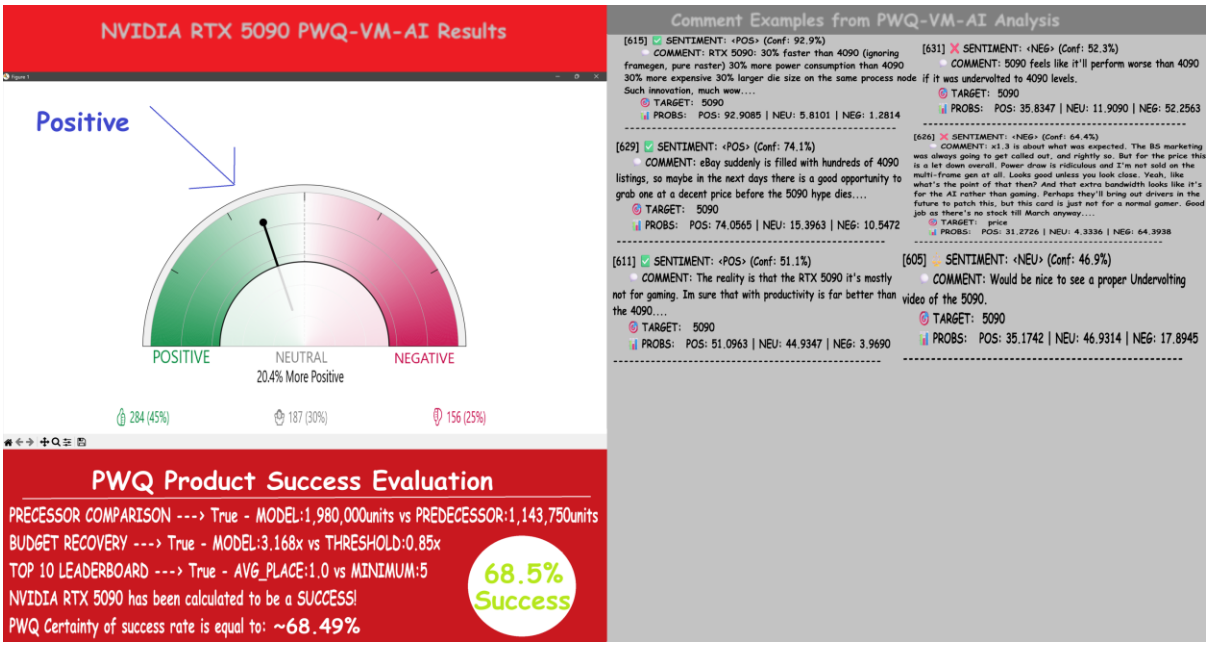
2. Using PWQ-VM-AI's intelligence, to extract sentiment from hundreds of opinions / comments from social media, then summing up and computing probability distribution, which gets plotted on VM-AI Barometer.

• Overall PWQ-VM-AI managed to successfully predict all three products' future glory, matching verdicts with the triple-area-evaluator module, across all samples. **However, despite the positive results, the final training accuracy of PWQ-VM-AI has reached 97.87% of accuracy, whilst on a real-world messy and hard-to-understand data, the accuracy is closer to ~74-78%.**

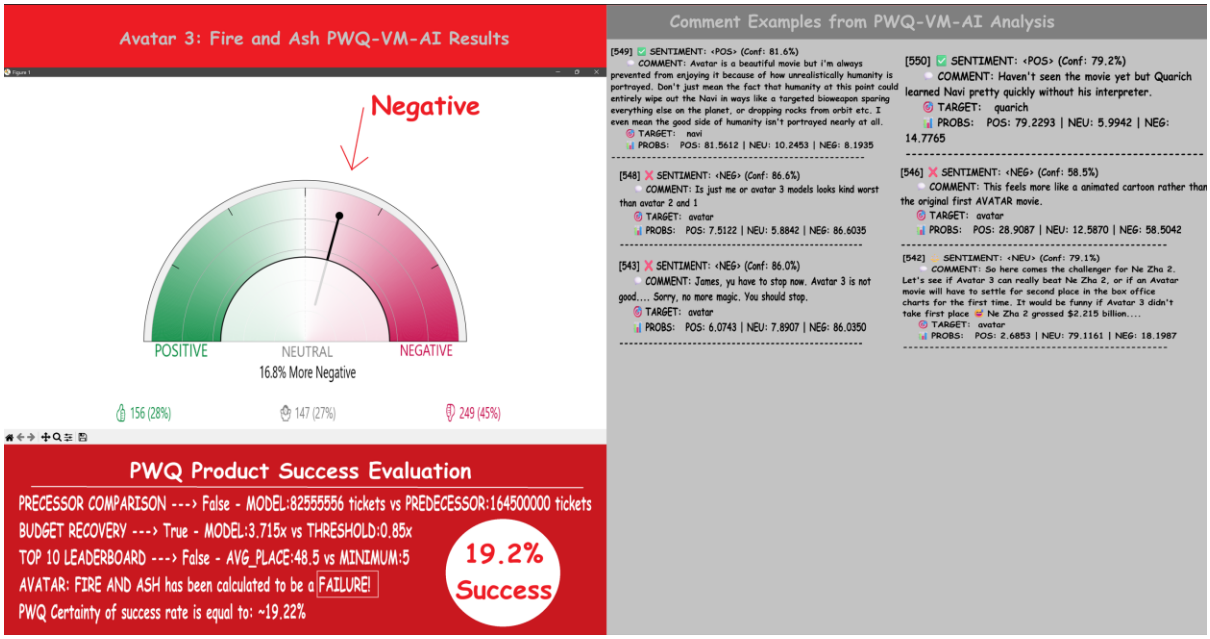
• iPhone 17 Pro Max -> 92.5% Success Rate | 6.7% Positivity of Sentiment



• NVIDIA RTX 5090 -> 68.5% Success Rate | 20.4% Positivity of Sentiment



• Avatar 3: Fire and Ash -> 19.2% Success Rate | 16.8% Negativity of Sentiment



IV. **Methodology and Real-World Impact.** By precisely mapping PWQ-VM-AI, the project demonstrates how AI can function as a practical tool within behavioral economics. The methodology includes:

• Data Scraping: Real-time collection of public data from financial and product-related discussion forums.

• Processing: Transformer-based analysis (PyTorch) to evaluate the "temperature" of public discourse.

• Visualization: Presentation of results through a clear emotional barometer (0–100%) alongside 3–5 representative comments that explain the contextual background and summarize user opinions on a given social media platform.

The project has direct real-world applications, offering an early-warning system for companies launching new products or analysts studying collective behavioral patterns.

V. **Ethics and Safety.** In compliance with the safety and ethics standards of the Acellus Science Fair, PWQ-VM-AI processes only publicly available data. All usernames and personal identifiers are removed during pre-processing stage, ensuring full anonymity and protection of individual privacy.

VI. **Competitive Landscape.**

• Brandwatch: AI-Powered social listening platform that monitors online conversations and performs sentiment analysis for brands, products and markets.

• Pulsar: A narrative intelligence platform that detects, maps, and tracks the evolution of social and media narratives across digital channels.

- **YouScan: A consumer intelligence platform that analyzes text and visual content from social media to identify emotions, trends, and brand perception.**
- **TipRanks: A financial analytics platform that aggregates analyst opinions, financial blogs, and market data to support investment decision-making.**
- **FinBERT: Domain-specific NLP model trained on financial text to classify sentiment and extract insights from financial documents and discussions.**

VII. **What makes PWQ-VM-AI Different.** PWQ-VM-AI does not merely classify sentiment as negative or positive, but quantifies the intesity and temporal dynamics of collective emotions expressed in public discourse. The model is trained on a distinct, purpose-built, fine-tuned dataset, contructed specifically to capture emotional escalation and saturation, rather than standard sentiment label. Additionally, the system applies a novel data-processing pipeline, that combines NRoPE's rotational embedding, PON's Paramater Initialization, Adaptive Dropout Layers with adjustable parameter per layer, per tensor, and a customized FFN architecture with enriches processed data, all to detect textual emotions, making it fundamentally different from conventional sentiment analysis solutions.

Aspect	Industry Standard	**PWQ-VM-AI V3S**
Sentiment Analysis	Binary positive/negative	**Emotional Intensity (0-100%)**
Target Awareness	Whole-document analysis	**Token-specific targeting**
Positional Encoding	Standard RoPE/Sinusoidal	**NRoPE with angle neutralization**
Dropout Mechanism	Fixed rate (0.1-0.5)	**Adaptive, learnable per layer**
Weight Initialization	Kaiming/Xavier/Orthogonal	**PON (Perturbated Ortho-Normalized)**
Architecture	Pre-trained (BERT, etc.)	**From-scratch custom Transformer**
Output	Classification only	**Barometer visualization + probabilities**
Efficiency	8-10GB at batch 32	**7.7GB at batch 524 (~19x more efficient)**

VIII. Potential Directions for Project Development.

- **VIII.I Technical Development.** Firsly, accuracy in detection of advanced emotions such as: irony, sarcasm and mockery, needs to improved. For that PWQ-VM-AI V4 will use Emoji & Punctuation Encoding, Contrastive Learning, Sentiment Discrepancy Detection, amongst others. Next, is to increase capacity of knowledge. The model needs to have: more layers, more token capability, a broader and more sophisticated dataset, and needs to be able to learn modern internet slang such as: idc, idk, icl, imo, etc. Lastly, since the future of PWQ-VM-AI is mostly centered on financial sentiment and market indication, the model would need to learn how to determine if a given market is bearish and bullish, and how that information, could be used as a indicator for investment decision-making strategies.

- **VIII.II Substantive Development.**

• Cross-Domain Applications: Building on its ability to quantify emotional intensity and detect emotional extremes, PWQ-VM-AI can be applied beyond financial markets to other products, services, and domains such as political discourse, public policy evaluation, marketing effectiveness, and consumer trust analysis, functioning as a behavioral early-warning system.

• Multimodal Emotion Analysis: The project can be extended from text-based analysis to a multimodal framework by integrating audio and video signals, including vocal stress patterns, speech rhythm, facial expressions, and micro-movements, enabling a deeper and more robust detection of emotional states across different communication channels.

• Emotional Coherence and Truthfulness Indicators: By combining emotional intensity, internal consistency of statements, and cross-modal signal alignment, the system may evolve toward an Emotional Coherence Index capable of identifying emotional instability, manipulation patterns, and potential deviations from truthful communication, without relying on semantic content alone.

IX. Logging Diary.

```
1. January 28th | Started with Token Embedding, LayerNorm, Linear Projection and
GAP. Learned the structure of pytorch's nn.Module inheritable class.
2. January 30th | Implemented Multi-Headed Self-Attention, using the famous QKV To-
ken projection. Had slight issues with full comprehension of dimen-
siones of the 4-5D tensors, but I've made it through.
3. February 3rd | Created the Feed-Forward Network (FFN) Layer, which consisted of
2 linear projection layers, one which expands the dimension allowing
activation to "capture" more  and higher-quality data, and one
reduces the tensor back to original shape, however with the higher-
quality data.
```

4. February 5th | It was time for Residual Additions, since without them, data that travels through deep models, eventually losses its original "meaning".
Residual Additions add the un-augmented data to the augmented one, ensuring meaning/values are preserved.
5. February 6th | I added a activation library, since I wanted to learn how each one works "under the hood". Currently there is over 22 distinct activations
6. February 7th | I engineered a special mix of 2 activations; PReLU & Mish. I liked the stability and smoothness of mish, and wanted to find out how It'd function with a learnable parameter from PReLU. Turns out it's fanta-stic.
7. February 8th | I added a multi-layer support for the transformer's "brain". All I did was create multiple instances of the necessary blocks per layer, appended them to a shared list, and in forward() just looped thru all layers calling the instances one-by-one.
8. February 9th | I added the HF Token / .env File Reading since my API was polling too much data. In addition fixed some tiny errors, and decided to crown PMish as PWQ-VM-AI's default activation.
9. February 19th | I spent some time researching initialization methods for weights, and found Orthogonal Init the best in performance, however, as my tradition I always like to slightly improve / tweak a concept I will use in my code.
So I decided to stop to a halt, and learn more about Linear Algebra / Matrices.
10. February 22nd | After going through 3b1b's Introduction to Linear Algebra course on You-
Tube, and countless hours studying matrices, I know understand the backbone logic behind Orthogonal initialization, and also its flaws.
11. February 26th | I have engineered a higher-performing alternative to orthogonal, which research and study shows outperforms it on diffrent architecture designs, demonstrates superior convergence speed, exceptional stability, and near-optimal final loss – statistically outperforming both Kaiming and Orthogonal initializations.
What I did is i added a slight perturbationmatrix to the Q matrix (derived from QR Decomposition using Householder Ref-lections algortihm), which still sort of maintains it's perfect 90deg angles, yet breaks that perfect symmetry.
12. February 28th | I have embedded the PON (Perturbated Ortho-Normalized) initializer to VM-AI
, and last step is to implement training loop. For that VM AI V1POC will utilize single-batch pytorch's graph and cross-entropy loss function to train the network to ~95%+.
13. March 2nd | After implementing the loop, and gathering a dataset from Kaggle, the network
has achieved ~98.52% accuracy on the training data, which is very great results. However, It didn't learn to generalize (ability to work with data excluded from dataset), therefore in the next versions I will: find a larger, more diversified dataset, add Dropout layers, and embed noise into token and positional embedding.
14. March 6th | VM AI V2R have introduced Target Awareness, which allows the network to tell
the diffrence between saying "A is better than B" and "B is better than A", since from POV of A the first sentence is <POS>, but second one is <NEG>. Secondly, we introduced NRoPE V1, which is a enhanced version of RoPE (Rotational Positional Encoding). NRoPE fixes the issue of creating infinitely large angles, which during extensive training, could hypothetically break / mess up the graph. NRoPE V1, fixes this by neutralizing the angle (by using modulo) to its safe range of [0, 2π].
15. March 7th | After augmenting the training loop, some fixes of NRoPE and PON, and minor changes to the architecture, I have acquired a dataset from HuggingFace, widely renowed as SemEval 2014.
16. March 10th | I have made some last changes and I introduce PWQ VM AI V3S, with whom I bring
NRoPE V3, a even more sophisticated version of V1. V3S brings custom Adaptive Drop-

out layers, which act as an vaccine, purposefully killing x% of neurons every cycle,
to teach the network not to rely on a single neuron, where x is fixed value.
I have also considered using Disentengled Attention from Microsoft's
ReBERTa transformer, however after research I found out it wouldn't function with NRoPE.
So I made a bit of both, dividing query token and key token into two separate matrices,
one spatial positioning and one content part, applying NRoPE with ARM parameter set to 2π,
instead of 2/3π used in embedding, later, stacking both matrices back together. Next upgrade
is Noise Addition, to positional and token embedding, to ensure they have a slight variation
and have to adapt more. Lastly, I have created a Mutli-Batched Training Loop which allows
for mutliple sentences to be trained at once, and on my 8GB VRAM laptop GPU, at 520 batch count
(which 520 sentences being processed at once), it only reached 7.7GB/8GB, which is very great.
This batched training, allowed for an epoch to be processed within a ~second which implies the
model is learning ~1040sentences/sec or 2batches of 520/ sec.
17. March 11th | I made some final touches towards Inference Loop, and training loop, fixed some bugs,
with weight loading, and got ready to train the V3S.
18. After ~8 minutes of training and ~270 epochs, of the SemEval 2014, dataset V3S reached,
94.09% which is slightly less than V2R however, we utilized Dropout which sort of enhances
training knowledge, but doesn't permit for the net to reached higher accuracy.
19. March 11th | I ran some inference tests, and it works exactly as i wanted. For sentence: "I hate iphone,
Samsung is better", with target=iphone, the sentiment is indeed negative. However, same sentence,
target=samsung, it gives back positive.
20. March 12th | I downloaded some youtube comments about product's release using GoogleAPI, and ran through
VM AI V3S, and after evaluating its results, it is ~78% accurate on sarcastic, messy, half-under-
standle youtube comments, which in the future developtment of VM AI, I would like to counter this
issue, but for now I'm content.
21. March 13th | I made a barometer script which allows to visualize, the results of VM AI in a nice, compact
manner, instead of numbers inside of the console. It's customizable, in any aspect, and it utilizes
matplotlib python library for the design.
22. March 14th | I ran PWQ-VM-AI V3S through all the comments, and collected the results which turned out to match the previously
evaluated product success verdicts across all 3 products.